Pandas

Python — Series, DataFrames, Filtering, Grouping & Merging

1 · pd.Series — 1-D Labelled Array

A **Series** is a one-dimensional labelled array that can hold any data type. Each element has an associated index label (default: 0, 1, 2 …).

```
import pandas as pd

s = pd.Series([10, 20, 30])
# 0    10
# 1    20
# 2    30
# dtype: int64

# Custom index
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
s['b']   # 20
```

■ A Series is like a single column of a DataFrame. It always has an index — by default integers starting from 0.

2 · pd.DataFrame — From Dictionary

The most common way to create a DataFrame is from a **dictionary**, where keys become column names and values become column data.

```
data = {
    'Name': ['Akash', 'Rohi'],
    'Age' : [25, 30]
}

df = pd.DataFrame(data)
#    Name  Age
# 0  Akash   25
# 1  Rohi    30
```

3 · pd.DataFrame — From List of Lists

You can also create a DataFrame from a **list of lists**, supplying column names separately.

```
df = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])
#    A  B
# 0  1  2
# 1  3  4
```

■ Each inner list becomes one row. The columns parameter assigns header names; omitting it gives integer column labels (0, 1, 2 …).

4 · Reading Data — read_csv() & read_excel()

```
# Read a CSV file
df = pd.read_csv('titanic.csv')

# Read an Excel file
df = pd.read_excel('data.xlsx')

# Useful optional parameters
pd.read_csv('file.csv', index_col=0)      # use first column as index
pd.read_csv('file.csv', nrows=100)         # read only first 100 rows
pd.read_csv('file.csv', usecols=['A','B']) # load specific columns only
```

■ Pandas supports many file formats: read_csv, read_excel, read_json, read_sql, read_parquet, and more. Always inspect the DataFrame with head() after loading.

5 · Exploring a DataFrame

```
df.describe()   # summary statistics for numeric columns
df.head()       # first 5 rows  (pass n for different count)
df.tail()       # last  5 rows
df.shape        # (rows, columns) — attribute, not a method
df.columns      # Index of column names
df.info()       # dtypes, non-null counts, memory usage
```

Attribute / Method	Returns	Note
df.describe()	Stats (mean, std, min, max…)	Numeric cols only by default
df.head(n)	First n rows (default 5)	Great for quick inspection
df.tail(n)	Last n rows (default 5)	Check for trailing data issues

df.shape	(rows, cols) tuple	Attribute — no parentheses
df.columns	Index of column names	Iterable; use .tolist()
df.info()	Dtypes + null counts	Reveals missing data quickly

6 · Column-Level Methods — unique, nunique, value_counts

Apply these on a **single column** (Series) to understand its distribution.

```
df['Gender'].unique()        # array of distinct values
df['Gender'].nunique()       # count of distinct values (number)
df['Gender'].value_counts()  # frequency count of each value

# value_counts() example output:
# male      577
# female    314
# Name: Gender, dtype: int64
```

■ These three methods — unique / nunique / value_counts — are your first tools for understanding a categorical column. value_counts() sorts by frequency descending by default.

7 · Filtering Rows — Boolean Indexing

Filter rows by passing a boolean condition inside square brackets. Combine multiple conditions with & (and) or | (or).

```
# Single condition
salesdata[salesdata['Subcategory'] == 'Bike Racks']

# Two conditions combined with & (each condition in parentheses)
salesdata[(salesdata['Subcategory'] == 'Bike Racks') &
          (salesdata['Subcategory2'] > 15)]

# Derived column – compute Profit Percentage on the fly
salesdata['Profit Percentage'] = (salesdata['Profit'] /
                                  salesdata['Revenue'])
```

Operator	Meaning	Example
&	AND — both conditions true	(df['A']>1) & (df['B']<5)
|	OR — at least one true	(df['A']>1) | (df['B']<5)
~	NOT — inverts the condition	~(df['A']>1)
.isin([])	Value is in a list	df['A'].isin([1,2,3])

■ Always wrap each condition in parentheses when combining with & or |. Python operator precedence would otherwise cause unexpected results.

8 · drop_duplicates()

```
df.drop_duplicates()                    # remove fully duplicate rows
df.drop_duplicates(subset=['Name'])     # deduplicate on specific column
df.drop_duplicates(keep='last')         # keep last occurrence instead of first

# To apply in-place:
df.drop_duplicates(inplace=True)
```

■ By default drop_duplicates() keeps the first occurrence and removes the rest. Use keep='last' or keep=False (drop all) as needed. Always check df.duplicated().sum() first to know how many rows will be removed.

9 · df.iloc — Integer-Location Based Indexing

iloc selects rows and columns by their **integer position** (like Python list indexing).

```
df.iloc[353:808, 0:6]
#        ^------^  ^--^
#        rows       columns  (both by integer position)

df.iloc[0]         # first row as a Series
df.iloc[0, 2]      # value at row 0, column 2
df.iloc[:, 0]      # entire first column
df.iloc[-1]        # last row
```

10 · df.loc — Label-Based Indexing

loc selects rows and columns by their **label / name**. Unlike iloc, the stop value is **inclusive**.

```
df.loc['a']              # row with index label 'a'
df.loc['a', 'Name']     # value at row 'a', column 'Name'
df.loc[:, 'Age']        # entire 'Age' column
df.loc[0:5, 'Name':'Age']  # rows 0-5, columns from Name to Age (inclusive)
```

	iloc	loc
Selects by	Integer position	Label / index name
Stop inclusive?	No (like Python slicing)	Yes
Column access	Column number	Column name
Use when	You know the position	You know the label

11 · groupby() — Aggregate Operations

groupby() splits the DataFrame into groups, applies an aggregate function, and combines the results — the classic **split** → **apply** → **combine** pattern.

```
# Total revenue by country
df.groupby('country')['Revenue'].sum()

# Multiple aggregations at once
df.groupby('country')['Revenue'].agg(['sum', 'mean', 'count'])

# Group by multiple columns
df.groupby(['country', 'Category'])['Revenue'].sum()

# Reset index to get a flat DataFrame back
df.groupby('country')['Revenue'].sum().reset_index()
```

Aggregate Function	Description
sum()	Total of the group
mean()	Average of the group
count()	Number of non-null entries
min() / max()	Minimum / maximum value
std()	Standard deviation
agg([])	Apply multiple functions at once

■ groupby() returns a GroupBy object. You must chain an aggregate function (sum, mean, count, etc.) to get a result. Use .agg() to apply several aggregations simultaneously.

12 · pd.merge() — Joining DataFrames

pd.merge() combines two DataFrames like a SQL JOIN, matching rows on a common key column.

```
# Basic merge on a key column
pd.merge(df1, df2, on='id', how='inner')

# how= controls the join type:
# 'inner' — only matching rows (default)
# 'left'  — all rows from df1, matching from df2 (NaN if no match)
# 'right' — all rows from df2, matching from df1
# 'outer' — all rows from both (NaN where no match)

# Merge on columns with different names
pd.merge(df1, df2, left_on='id', right_on='emp_id', how='left')
```

how=	SQL Equivalent	Rows Kept
'inner'	INNER JOIN	Only rows with matching key in both
'left'	LEFT JOIN	All of df1; matched rows from df2
'right'	RIGHT JOIN	All of df2; matched rows from df1
'outer'	FULL OUTER	All rows from both; NaN where no match

■ Think of pd.merge() as a SQL JOIN. Use 'inner' when you only want records present in both tables. Use 'left' when df1 is your primary table and you want to enrich it with data from df2 without losing any rows from df1.

Quick Reference — Pandas Essentials

Method / Attribute	Purpose	Note
pd.Series([])	1-D labelled array	Single column of data
pd.DataFrame({})	2-D table from dict or list	Keys → column names
pd.read_csv()	Load CSV into DataFrame	Many optional params
pd.read_excel()	Load Excel into DataFrame	Needs openpyxl
df.describe()	Summary stats	Numeric cols only
df.head() / tail()	First / last n rows	Default n=5
df.shape	(rows, cols)	Attribute, no ()
df.info()	Dtypes + null counts	Good for data audit
df['col'].unique()	Distinct values	Returns array
df['col'].nunique()	Count of distinct values	Returns integer
df['col'].value_counts()	Frequency table	Sorted descending
df[condition]	Filter rows	Use & | ~ for logic
df.drop_duplicates()	Remove duplicate rows	keep='first'/'last'
df.iloc[r, c]	Select by integer position	Stop is exclusive
df.loc[r, c]	Select by label	Stop is inclusive
df.groupby(col)	Group + aggregate	Chain .sum() etc.
pd.merge(df1, df2)	Join two DataFrames	how='inner/left/…'

```
# End-to-end Pandas pipeline example
import pandas as pd

df = pd.read_csv('sales.csv')
df.drop_duplicates(inplace=True)

# Filter: only Bike Racks with Subcategory2 > 15
filtered = df[(df['Subcategory'] == 'Bike Racks') &
              (df['Subcategory2'] > 15)]

# Derived column
filtered['Profit Percentage'] = filtered['Profit'] / filtered['Revenue']

# Group and aggregate
summary = filtered.groupby('Country')['Revenue'].sum().reset_index()
print(summary)
```

■ This pipeline — load → clean → filter → enrich → aggregate — covers 80% of real-world Pandas work. Always check df.info() and df.describe() right after loading data.